

Sistema de Agendamento de Consultas Médicas

Um sistema CRUD completo de agendamento de consultas médicas que demonstra conceitos fundamentais de **Sistemas Operacionais**, desenvolvido como trabalho para a disciplina de S.O.

Link para o Repositório

<https://github.com/gabrieldsn2006/Sistema-de-Agendamento>

Visão Geral

Este projeto implementa uma aplicação full-stack para gerenciamento de agendamentos médicos, integrando conceitos-chave de Sistemas Operacionais como concorrência, sincronização, gerência de memória, manipulação de arquivos e I/O assíncrono.

Tecnologias

- **Backend:** Python, FastAPI, SQLAlchemy, SQLite/aiosqlite
 - **Frontend:** HTML5, JavaScript (Vanilla), Bootstrap, Axios
 - **Principais Bibliotecas:** asyncio, threading, reportlab (PDF), pathlib
-

☐ Quick Start

Frontend

```
cd frontend
npm install axios bootstrap
npm run dev
```

Acesse: <http://localhost:5173>

Backend

```
cd backend
python -m venv venv

# Windows
venv\Scripts\activate

# Linux/macOS
source venv/bin/activate

# Instalar dependências
pip install fastapi uvicorn sqlalchemy aiosqlite python-multipart reportlab
python-dotenv
pip install "fastapi[standard]"
pip install tzdata

# Iniciar servidor
uvicorn app.main:app --reload
```

```
# ou
python run.py
```

API disponível em: <http://localhost:8000>

Documentação interativa: <http://localhost:8000/docs>

□ Estrutura do Projeto

```
Sistema-de-Agendamento/
├── backend/
│   ├── src/
│   │   ├── concorrencia/
│   │   │   ├── locks.py          # □ Sincronização (mutex/semáforo)
│   │   │   └── workers.py       # □ Threads e escalonamento
│   │   ├── config/
│   │   │   ├── database.py      # □ Sistema de Arquivos
│   │   │   ├── platform_info.py # □ Informações do S0
│   │   │   ├── logger.py        # □ Logging e dispositivos
│   │   │   └── settings.py      # □ Configuração por S0
│   │   ├── core/
│   │   │   ├── models/         # Modelos de dados
│   │   │   ├── repositories/   # Acesso a dados
│   │   │   ├── routers/        # Endpoints REST
│   │   │   └── services/       # Lógica de negócios
│   │   ├── reports/            # □ Geração de relatórios (I/O)
│   │   ├── storage/            # □ Cache e armazenamento
│   │   └── main.py              # Ponto de entrada
│   ├── data/
│   │   ├── backups/            # □ Backups do banco
│   │   ├── cache/              # □ Cache temporário
│   │   └── logs/                # □ Arquivos de log
│   └── requirements.txt
├── frontend/
│   ├── index.html
│   ├── consultas.html
│   ├── medicos.html
│   ├── relatorios.html
│   ├── src/
│   │   ├── api/
│   │   ├── pages/
│   │   ├── utils/
│   │   └── main.js
│   └── package.json
└── README.md
```

Arquitetura

Backend (API REST - FastAPI)

A arquitetura segue o padrão **Clean Architecture** com separação de responsabilidades:

Route → Service → Repository → Database

Componentes:

- **Routers:** Endpoints REST (/appointments, /doctors, /patients, /reports, /system)

- **Services:** Lógica de negócios
- **Repositories:** Camada de acesso a dados (SQLAlchemy)
- **Models:** Entidades do banco de dados
- **Schemas:** DTOs (Data Transfer Objects) para validação

Frontend (SPA - Single Page Application)

Interface web responsiva com:

- **Pages:** Componentes para cada seção (consultas, médicos, pacientes, relatórios)
- **API Client:** Módulo de comunicação HTTP (axios)
- **Utils:** Utilitários de notificações e helpers

☐ Conceitos de Sistemas Operacionais Demonstrados

1. Sincronização e Exclusão Mútua (concorrença/locks.py)

Problema: Race condition quando duas requisições tentam agendar consultas para o mesmo médico no mesmo horário.

Solução:

```
# asyncio.Lock – equivalente a mutex/semáforo binário do kernel
async with appointment_lock():
    # Seção crítica: check conflict → create appointment
    if await repo.doctor_has_conflict(...):
        raise HTTPException(status_code=409, ...)
    appointment = await repo.create(data)
```

Conceitos S.O.:

- **Exclusão Mútua (Mutual Exclusion):** Apenas uma corrotina acessa a seção crítica
- **Semáforo Binário:** asyncio.Lock implementa mutex no espaço do usuário
- **Seção Crítica:** Bloco de código que acessa recurso compartilhado
- ⚠ **Limitação:** Em produção com múltiplos workers (Gunicorn multiprocess), seria necessário lock distribuído (Redis/Redlock)

2. Threads e Escalonamento (concorrença/workers.py)

Problema: Operações de I/O pesadas (backup, limpeza de cache) bloqueiam requisições.

Solução:

```
# Thread daemon para background tasks
_worker_thread = threading.Thread(
    target=_worker_loop,
    daemon=True # Encerra com o processo principal
)
_worker_thread.start()
```

```
# Fila thread-safe (produtor/consumidor)
_task_queue = queue.Queue(maxsize=50)
```

Conceitos S.O.:

- **Threads:** Múltiplas threads de execução no mesmo processo
- **Escalonamento Preemptivo:** SO alterna entre threads (Linux)
- **Daemon Threads:** Threads que encerram com o processo principal
- **Produtor/Consumidor:** Padrão clássico de SO para comunicação thread-safe
- **Bloqueio em I/O:** SO suspende thread durante I/O, acorda quando dados disponíveis

3. Sistema de Arquivos (config/database.py, concorrência/workers.py, reports/)

Operações:

```
# Backup com preservação de metadados (inode)
shutil.copy2(db_path, dest) # preserva atime, mtime, permissions

# Limpeza de cache baseada em timestamps
for f in cache_dir.iterdir():
    if (now - f.stat().st_mtime) > 3600:
        f.unlink()

# Criação de estrutura de diretórios (agnóstico de plataforma)
backup_dir = Path("data/backups")
backup_dir.mkdir(parents=True, exist_ok=True)
```

Conceitos S.O.:

- **Inode:** Estrutura que armazena metadados do arquivo (timestamps, permissões)
- **Paths agnósticos:** `pathlib.Path` funciona em Windows, Linux, macOS
- **Syscalls de I/O:** `shutil.copy2` → read/write syscalls em bloco
- **Permissões:** Manipulação de bits de permissão via OS

4. Gerência de Memória (storage/cache.py)

Implementação:

```
# Cache com limite de tamanho e TTL (Time To Live)
class Cache:
    def __init__(self, max_size=100, ttl=3600):
        self.data = {}
        self.expiry = {}
        self.max_size = max_size
```

Conceitos S.O.:

- **Alocação Dinâmica:** Estruturas de dados crescem/encolhem em tempo de execução
- **TTL (Time To Live):** Limpeza automática evita vazamento de memória
- **Limite de Tamanho:** Controle de consumo de memória

5. Entrada/Saída Assíncrona (main.py, core/routers/)

Padrão:

```
# async/await – I/O não-bloqueante
@app.get("/appointments/")
async def list_appointments(repo: AppointmentRepository = Depends(_repo)):
    return await repo.get_all()

# Event loop do asyncio coordena múltiplas operações de I/O
```

Conceitos S.O.:

- **I/O Não-Bloqueante:** Operações de I/O não bloqueiam a execução
- **Event Loop:** Gerenciador de eventos (similar a epoll/kqueue do kernel)
- **Corrotinas:** Funções que podem ser suspensas e retomadas

6. Configuração Dependente do SO (config/platform_info.py)

Deteção automática:

```
import platform

os_name = platform.system() # "Windows", "Linux", "Darwin"
arch = platform.machine()   # "x86_64", "ARM64"
python_version = platform.python_version()

# Paths específicos do SO
BASE_DIR = Path(__file__).parent.parent.parent
if os_name == "Windows":
    DATA_DIR = Path.home() / "AppData" / "Local" / "Agendamento"
else:
    DATA_DIR = Path.home() / ".agendamento"
```

Conceitos S.O.:

- **Chamadas de Sistema (Syscalls):** platform.* chamam APIs do SO
- **Paths Específicos:** Localização padrão de dados por SO
- **Arquitetura:** Deteção de CPU (x86_64, ARM)

7. Logging e Gerência de Dispositivos (config/logger.py)

Implementação:

```
import logging

logger = logging.getLogger(__name__)
handler = logging.FileHandler("data/logs/app.log")
formatter = logging.Formatter(
    '%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)
```

Conceitos S.O.:

- **Dispositivos de I/O:** Escrita em arquivo/stdout

- **Buffer de I/O:** Logging usa buffer para otimizar escrita
- **Timestamp do Sistema:** `datetime.now()` sincroniza com relógio do SO

III Funcionalidades

Pacientes (Patients)

Operação	Endpoint	Descrição
Criar	POST <code>/patients/</code>	Cadastrar novo paciente
Listar	GET <code>/patients/</code>	Listar todos (paginado)
Buscar	GET <code>/patients/{id}</code>	Buscar por ID
Atualizar	PUT <code>/patients/{id}</code>	Atualizar dados
Deletar	DELETE <code>/patients/{id}</code>	Remover paciente

Médicos (Doctors)

Operação	Endpoint	Descrição
Criar	POST <code>/doctors/</code>	Cadastrar novo médico
Listar	GET <code>/doctors/</code>	Listar todos (paginado)
Buscar	GET <code>/doctors/{id}</code>	Buscar por ID
Atualizar	PUT <code>/doctors/{id}</code>	Atualizar dados
Deletar	DELETE <code>/doctors/{id}</code>	Remover médico

Consultas (Appointments) — ☆ Com Sincronização

Operação	Endpoint	Descrição
Criar	POST <code>/appointments/</code>	Agendar com exclusão mútua
Listar	GET <code>/appointments/</code>	Listar todas (paginado)
Buscar	GET <code>/appointments/{id}</code>	Buscar por ID
Por Médico	GET <code>/appointments/doctor/{id}</code>	Consultas de um médico
Por Paciente	GET <code>/appointments/patient/{id}</code>	Consultas de um paciente
Atualizar	PUT <code>/appointments/{id}</code>	Atualizar dados
Deletar	DELETE	Cancelar consulta

Operação	Endpoint	Descrição
	/appointments/{id}	

Validações:

- Impede agendamento de médico em dois lugares ao mesmo tempo
- Impede agendamento de paciente em dois lugares ao mesmo tempo
- Verifica existência de paciente e médico

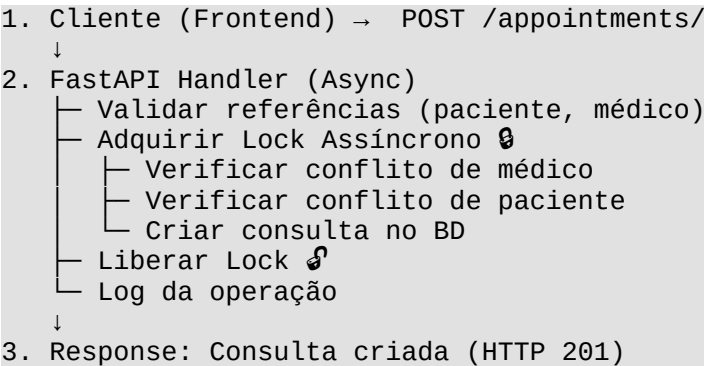
Relatórios

Tipo	Endpoint	Descrição
PDF	GET /reports/appointments/pdf	Relatório em PDF
CSV	GET /reports/appointments/csv	Relatório em CSV

Sistema (System Info)

Endpoint	Descrição
GET /info	Informações do sistema (SO, threads, locks)
POST /tasks/backup	Enfileira backup automático
GET /tasks/{id}	Status da tarefa background

 Fluxo de Agendamento (Destacando Sincronização)



Se dois agendamentos chegarem simultaneamente:

- Task A adquire lock → verifica → cria
- ⌚ Task B aguarda lock

- Task B acquire lock → detecta conflito → erro (HTTP 409)

Estrutura de Diretórios (Runtime)

```
data/
├── agendamento.db          # ▢ Banco SQLite (Sistema de Arquivos)
├── backups/
│   ├── agendamento_20250524_143022.db
│   └── agendamento_20250524_140015.db
├── cache/                 # ▢ Arquivos de cache (TTL cleanup)
│   └── cached_report_*.tmp
├── logs/                  # ▢ Logging por dispositivo
│   └── app.log
```

Configuração

Variáveis de Ambiente (.env)

```
# Backend
DATABASE_URL=sqlite+aiosqlite:///data/agendamento.db
LOG_LEVEL=INFO
CORS_ORIGINS=["http://localhost:5173", "http://localhost:3000"]

# Frontend
VITE_API_URL=http://localhost:8000
```

Configuração por SO

Detectada automaticamente em config/platform_info.py:

```
Windows → AppData/Local/Agendamento
Linux   → ~/.agendamento
macOS   → ~/Library/Application Support/Agendamento
```

Dependências

Backend

```
fastapi==0.109.0      # Web framework async
uvicorn==0.27.0       # ASGI server
sqlalchemy==2.0.24    # ORM
aiosqlite==0.19.0     # Driver SQLite async
python-multipart==0.0.6 # Form parsing
reportlab==4.0.7      # PDF generation
python-dotenv==1.0.0  # .env loader
```

Frontend

```
axios          # HTTP client
bootstrap      # CSS framework
```

❑ Testando Concorrência

1. Teste de Race Condition (Sem Lock)

Remover linha em `appointments.py`:

```
# async with appointment_lock(): # ❑ Comentar
# ...
```

Executar:

```
# Terminal 1
curl -X POST http://localhost:8000/appointments/ \
  -H "Content-Type: application/json" \
  -d '{"doctor_id":1, "patient_id":1, "scheduled_at":"2025-05-25T10:00:00"}'

# Terminal 2 (simultaneamente)
curl -X POST http://localhost:8000/appointments/ \
  -H "Content-Type: application/json" \
  -d '{"doctor_id":1, "patient_id":2, "scheduled_at":"2025-05-25T10:00:00"}'

# ❑ Resultado: Ambas as consultas são criadas (BUG!)
```

2. Teste de Race Condition (Com Lock) ❑

Com `async with appointment_lock()`:

```
# Executar os mesmos comandos simultaneamente
# ❑ Resultado: Uma consulta criada (201), outra rejeitada (409)
```

3. Verificar Logs

```
tail -f data/logs/app.log
```

```
# Output:
# [LOCK] Task 'create_appointment' aguardando o lock...
# [LOCK] Task 'create_appointment' adquiriu o lock.
# [LOCK] Task 'create_appointment' liberou o lock.
```

📊 Monitoramento

Endpoint /info

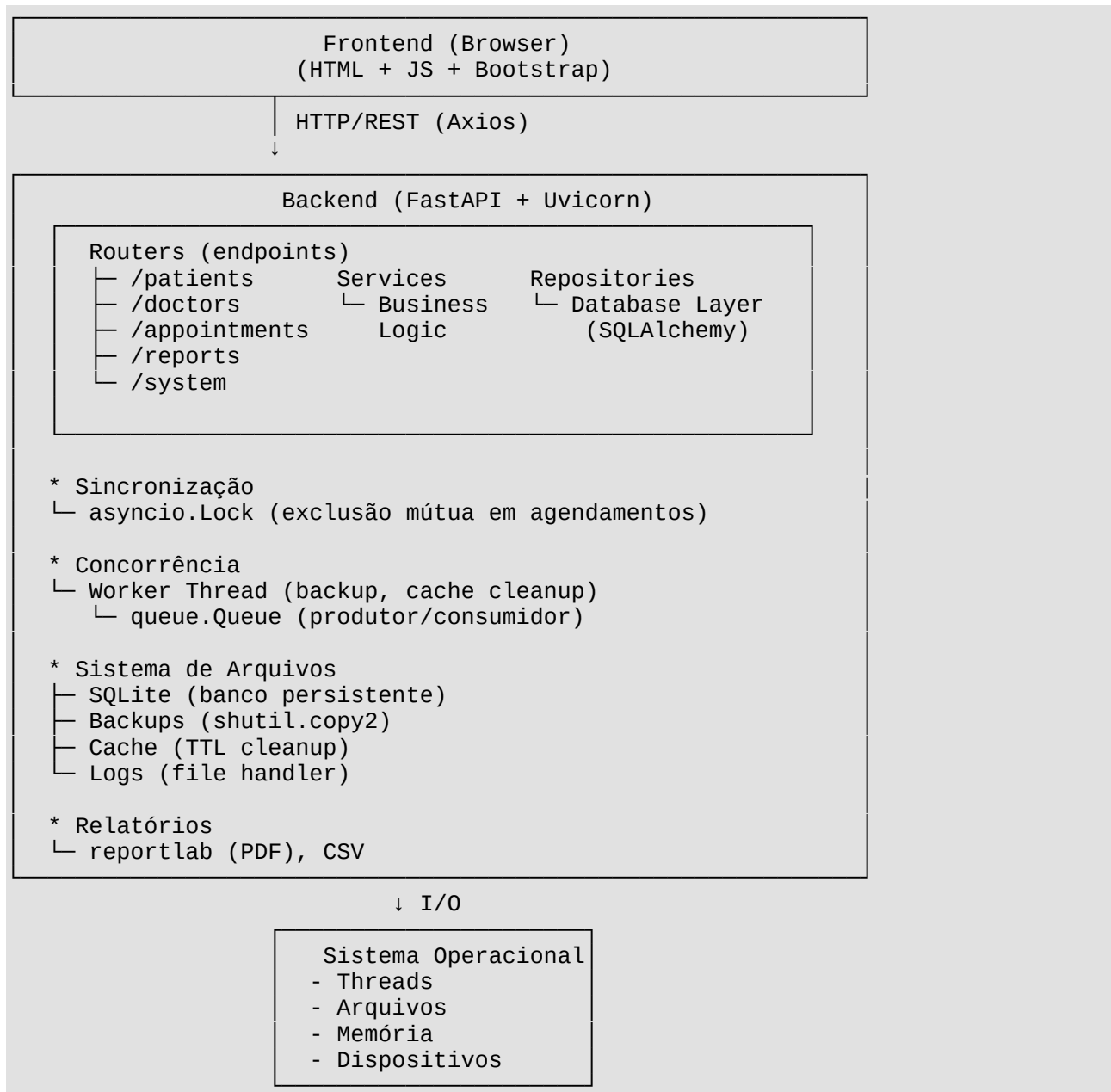
```
{
  "status": "online",
  "platform": {
    "system": "Windows",
    "machine": "x86_64",
    "python_version": "3.11.0"
  },
  "concorrenca": {
    "lock_type": "asyncio.Lock",
    "locked": false,
    "thread_count": 5,
    "current_thread": "MainThread"
  },
  "worker": {
```

```

    "thread_name": "BackgroundWorker",
    "thread_alive": true,
    "thread_daemon": true,
    "queue_size": 0,
    "completed_tasks": 3
  }
}

```

⚙ Diagrama de Componentes



☐ Notas Técnicas

Por que `asyncio.Lock` em vez de `threading.Lock`?

```
# □ threading.Lock bloquearia o event loop
with threading.Lock():
    # Durante essa espera, nenhuma outra requisição é processada
    # O server "congela" enquanto aguarda

# □ asyncio.Lock cede controle ao event loop
async with asyncio.Lock():
    # Event loop continua processando outras requisições
    # CPU não é desperdiçada enquanto aguarda
```

Por que `daemon=True` no Worker?

```
# daemon=True = thread encerra quando processo principal encerra
# Evita que o programa "trave" tentando aguardar a thread
_worker_thread = threading.Thread(..., daemon=True)
```

Limitação: Single-Process Lock

Gunicorn Worker 1 asyncio.Lock	Gunicorn Worker 2 asyncio.Lock	Gunicorn Worker 3 asyncio.Lock
--------------------------------------	--------------------------------------	--------------------------------------

(independente) (independente) (independente)

⚠ Problema: Cada processo tem seu próprio lock!
Dois workers podem agendar conflitantemente

□ Solução: Lock Distribuído (Redis/Redlock)

☐ Debugging

Ligar Modo Debug

```
LOG_LEVEL=DEBUG uvicorn app.main:app --reload

# Saída:
# [LOCK] Task 'create_appointment' aguardando o lock...
# [WORKER] Tarefa enfileirada: 'backup' id=a1b2c3d4
# [BACKUP] Backup criado: agendamento_20250524_143022.db (512 KB)
```

Verificar Status do Worker

```
curl http://localhost:8000/system/worker/status

# Response:
{
  "thread_name": "BackgroundWorker",
  "thread_alive": true,
  "queue_size": 0,
  "completed_tasks": 5,
  "active_threads_total": 3
}
```

Referências Conceituais

Conceito S.O.	Arquivo	Detalhes
Mutex/Semáforo	concorrenca/locks.py	Exclusão mútua em agendamentos
Threads	concorrenca/workers.py	Worker background daemon
Produtor/Consumidor	concorrenca/workers.py	queue.Queue
Sistema de Arquivos	config/database.py	Persistência SQLite + Backups
I/O Assíncrono	main.py, core/routers/	async/await, event loop
Gerência de Memória	storage/cache.py	Cache com TTL e limite
Logging/Dispositivos	config/logger.py	File handlers
Configuração de SO	config/platform_info.py	Platform detection

☐ Conclusão

Este projeto demonstra de forma prática como conceitos teóricos de Sistemas Operacionais são aplicados no desenvolvimento real de software:

- **Sincronização:** Evita race conditions em operações críticas
- **Concorrência:** Múltiplas tarefas rodando eficientemente
- **Persistência:** Dados organizados no sistema de arquivos
- **Performance:** I/O não-bloqueante com asyncio
- **Portabilidade:** Funciona em Windows, Linux e macOS

☐ Autor

Desenvolvido como trabalho da disciplina de **Sistemas Operacionais**

Data de Entrega: 27/05/2025

Status: Completo